

Embedded System

High Level Hardware Synthesis II

Mostafiz Ahammed

Lecturer

*Department of IoT and Robotics Engineering
University of Frontier Technology, Bangladesh.*

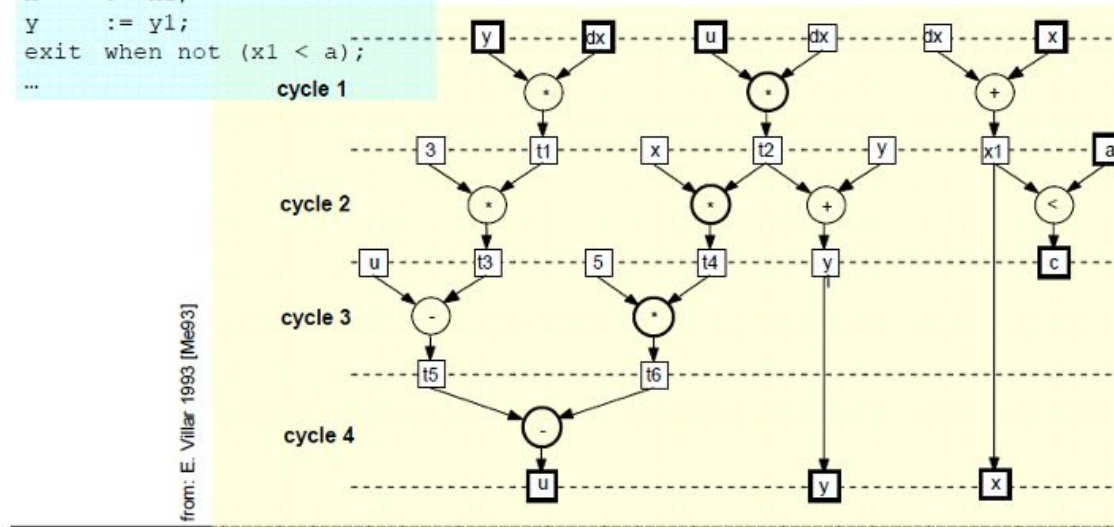
Topics

- Mapping Specification → HW by High Level Synthesis / Design (architecture synthesis for data-dominated systems)
 - High Level Synthesis Overview
 - Basic Scheduling Algorithms: ASAP, ALAP
 - Resource-Constrained Scheduling
 - Time-Constrained Scheduling
 - Advanced Scheduling: Operator Pipelining, Chaining & Multicycling
 - **Allocation, Binding, Data Path & Controller Generation**
 - **Pipelined Data Path Design**
 - **Pipeline Controller Design**

Example: Scheduling Result

```
x1 := x + dx;  
y1 := y + (u * dx);  
u := u - (5 * x * u *  
dx) - (3 * y * dx);  
x := x1;  
y := y1;  
exit when not (x1 < a);  
...
```

We will use this as an example to show the steps after scheduling



Let us assume that the schedule shown above has been derived for the differential equation solver algorithm. This schedule will be used in the following to demonstrate the further steps: **allocation**, **binding**, and **datapath / controller generation**.

Allocation

Allocation is the process of deciding **how many hardware resources** are needed to implement the scheduled operations.

What is allocated?

- Functional units (adders, multipliers, comparators)
- Registers
- Multiplexers
- Memories

Based on

- Scheduling result (ASAP / ALAP / Resource / Time constrained)
- Degree of parallelism
- Cost and performance constraints

Allocation

Example

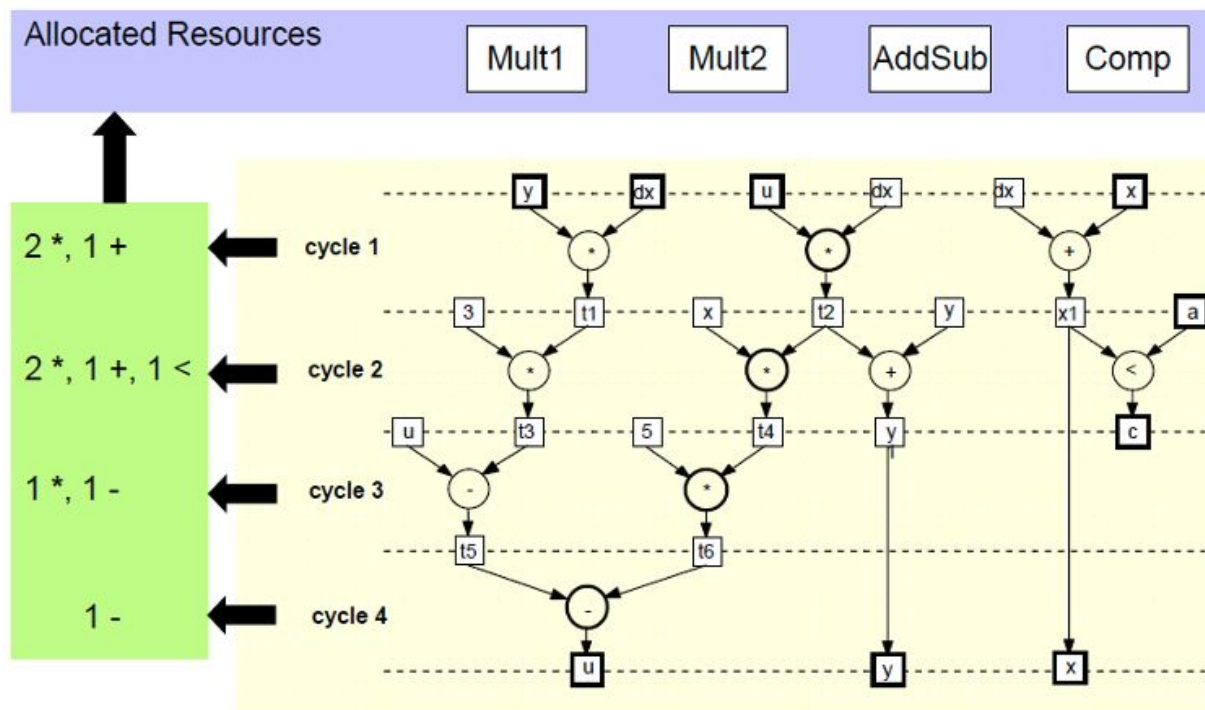
- If in one cycle:
- 3 additions occur in parallel → allocate **3 adders**
- 1 multiplication occurs → allocate **1 multiplier**

Goal

- Meet performance requirements
- Minimize hardware cost

Resource Allocation

Resource allocation in HLS determines the number of functional units of each type required to implement a given schedule by analyzing the maximum parallel usage of operations across all cycles, and to take these resources from a target library so that they can later be used for data path generation.



Binding

- ◆ **Binding** is the step in High-Level Synthesis (HLS) that decides:
 - ◆ **Which scheduled operation is executed on which allocated hardware resource.**
- ◆ **Binding** assigns each scheduled operation and variable to a **specific allocated hardware resource.**

In simple words:

- **Allocation** decides *how many* resources exist
- **Binding** decides *which operation uses which resource*

Binding with a Single Resource (No Choice Case)

❖ If **only one resource** is allocated for an operation type:

❖ Example:

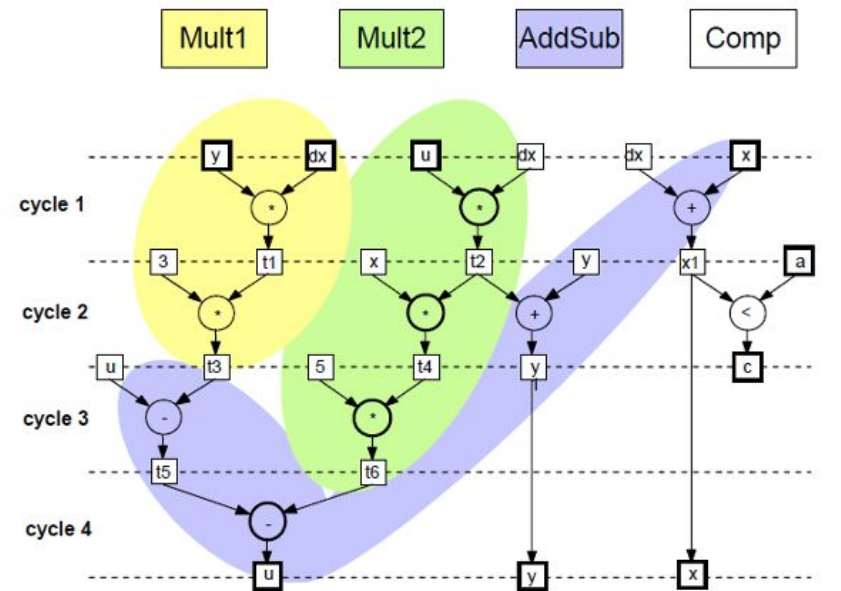
❖ One **adder/subtractor** for all $+$ and $-$ operations

👉 Then:

❖ There is **no flexibility**

❖ All additions and subtractions **must use that single resource**

❖ Binding is **fixed and obvious**



Binding with Multiple Resources (Choice Exists)

- ❖ If **multiple resources of the same type** are allocated:

- ❖ Example:

- ❖ Two multipliers: **Mult1** and **Mult2**

- ❖ Then:

- ❖ Different valid bindings are possible

- ❖ In **cycle 1**, two multiplications occur:

- ❖ Left multiplication $\rightarrow$ Mult1

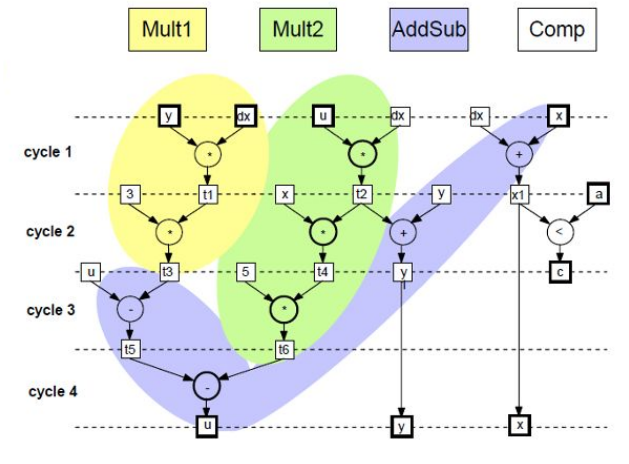
- ❖ Right multiplication $\rightarrow$ Mult2

OR

- ❖ Left $\rightarrow$ Mult2

- ❖ Right $\rightarrow$ Mult1

- ❖ Both are functionally correct, but **hardware cost and wiring differ**.

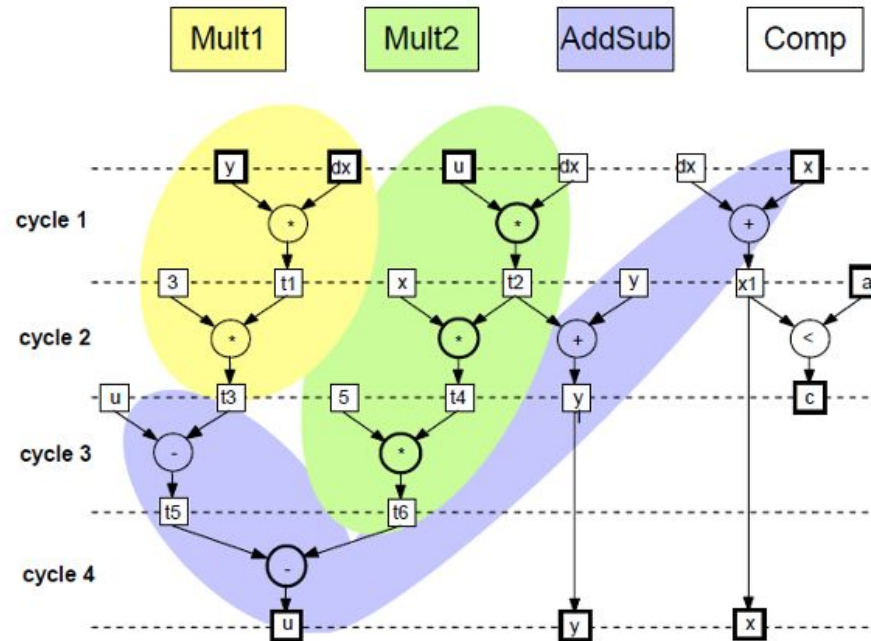


Binding

Binding defines which operators are implemented with which resources.

There are degrees of freedom, e.g. one of the $*$ operations bound to Mult2 could be bound to Mult1 instead.

Binding influences the wiring between resources.



❖ When **multiple resources of the same type exist**, binding is **not unique**.

- ❑ A $*$ operation currently bound to **Mult2**
- ❑ Could alternatively be bound to **Mult1**
- ❑ Both are functionally correct

❖ No need to transfer data between Mult1 $\leftrightarrow$ Mult2

❖ Fewer:

- ❑ Wires
- ❑ Multiplexers
- ❑ Control signals

Data Path Synthesis

- ❖ **Data Path Synthesis** is the step in HLS where the **actual hardware structure** that performs computations and moves data **based on the results of scheduling, allocation, and binding**.
- ❖ Datapath synthesis builds the hardware that does the math and moves the data.

What we know after each HLS step

- **After Scheduling**

We know *when* each operation executes (cycle-wise).

- **After Allocation**

We know *what hardware resources* exist (e.g., Mult1, AddSub).

- **After Binding**

We know *which operation uses which resource in which cycle*.

Once these are known, the **datapath can be built**.

Inputs to Data Path Synthesis

Datapath synthesis uses the outcomes of earlier HLS steps:

- **Scheduling**

- ✓ Decides *when* each operation executes (cycle-wise)

- **Allocation**

- ✓ Decides *how many* hardware resources exist

- **Binding**

- ✓ Decides *which operation uses which resource*

These three together **fully define** what the datapath must look like.

Datapath

The synthesized Datapath typically includes:

1. **Functional Units**
2. **Registers**
3. **Multiplexers (MUXes)**
4. **Interconnections (Wires / Buses)**

Functional Units

- Adders / Subtractors
- Multipliers
- Comparators
- ALUs

Registers

- Store input values
- Store intermediate (temporary) results
- Hold outputs between cycles

Multiplexers (MUXes)

- Select inputs to functional units
- Needed when:
 - A resource is shared across cycles
 - Multiple values can feed the same unit

Interconnections (Wires / Buses)

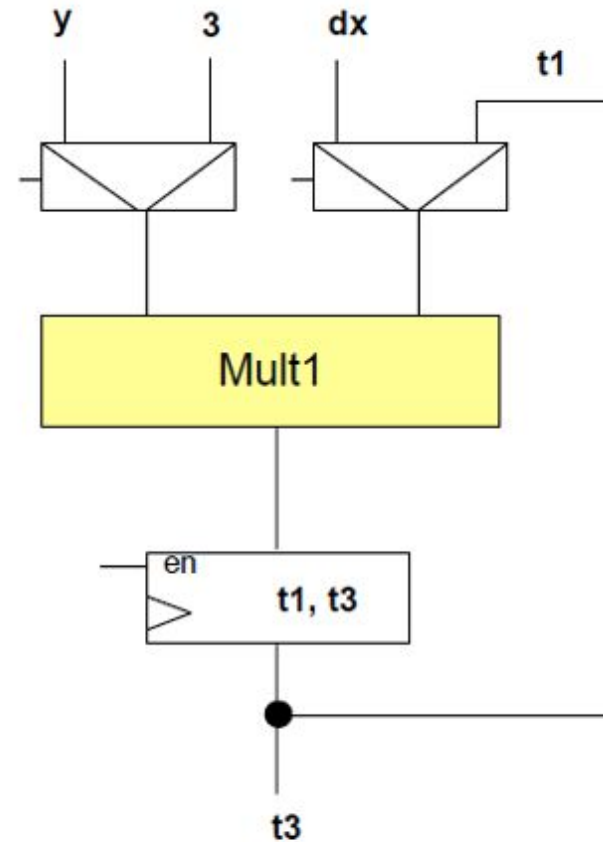
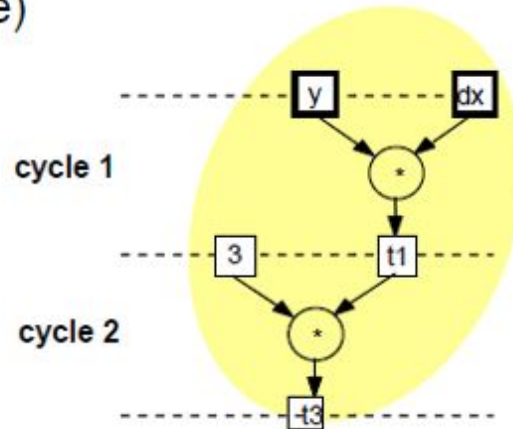
- Connect registers to functional units
- Connect functional units back to registers
- Determined mainly by **binding decisions**

Data Path Synthesis

Connection of
resources

Multiplexers to
select between
multiple possible
operands

Registers for
storage of
(intermediate)
results

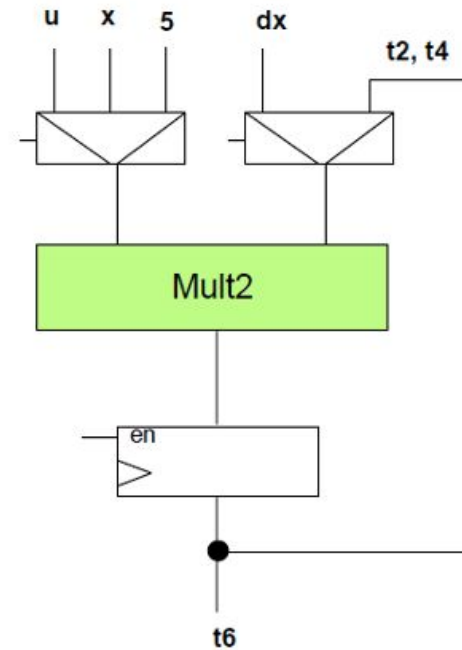
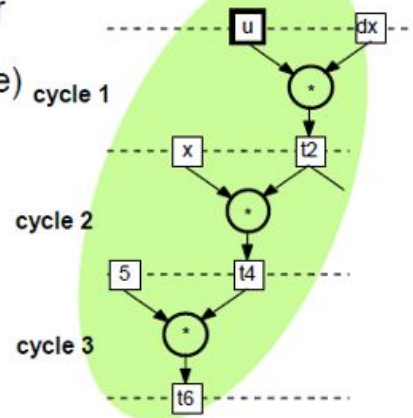


Data Path Synthesis (Cont'd)

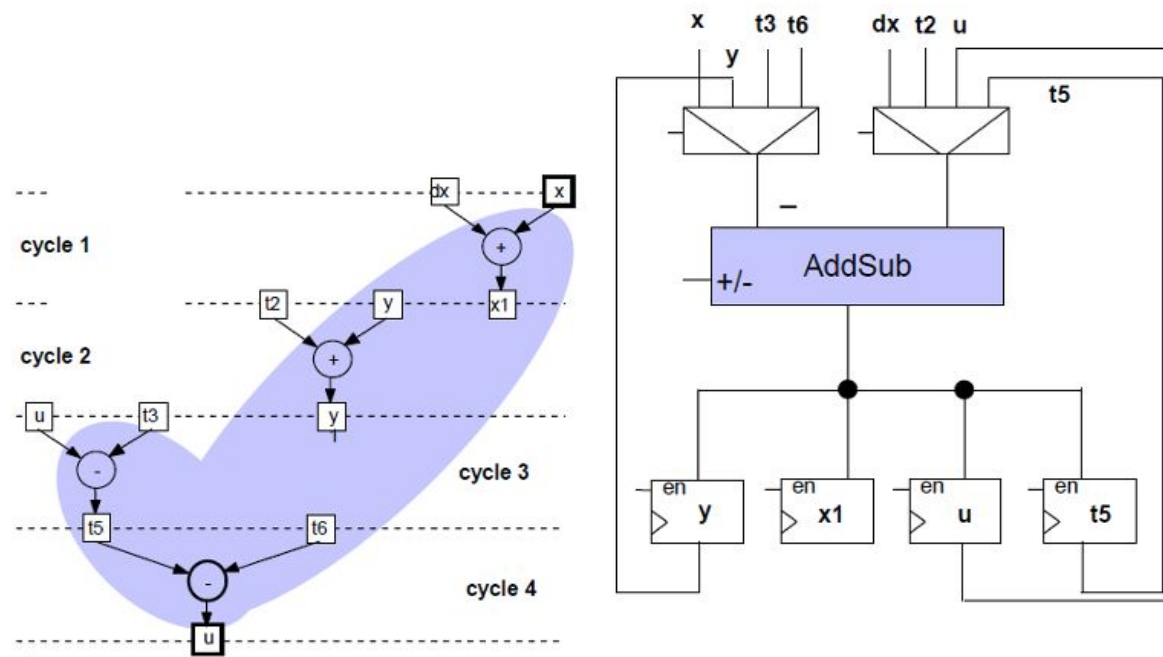
Connection of resources

Multiplexers to select between multiple possible operands

Registers for storage of (intermediate) results



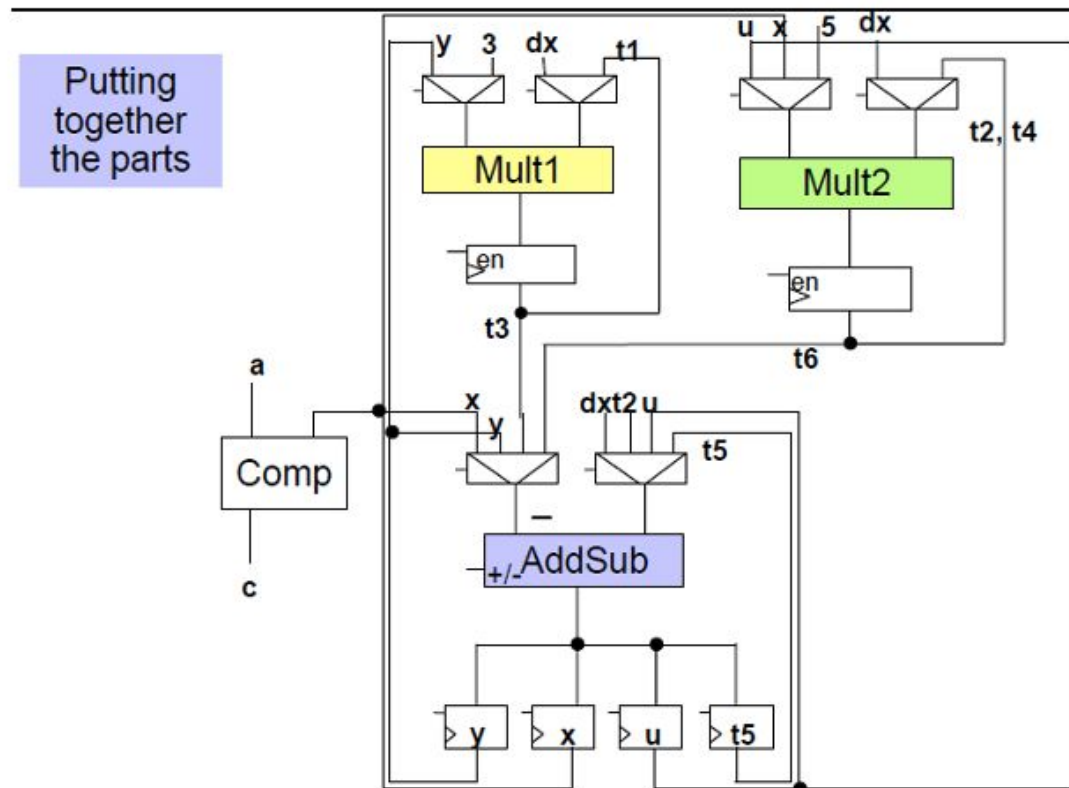
Data Path Synthesis (Cont'd)



We need more than one register to store the results of the computation. The reason is that the lifetime of these operands (y, x1, u, t5) is overlapping. The lifetime is the set of cycles during which the operand must be stored.

Therefore, we cannot use the same register to store all operands.

Data Path Synthesis (Cont'd)



After generating sub-datapaths for each resource, these sub-datapaths are connected together, again according to the data flow specified by the DFG.

The slide shows the **resulting complete data path** for our example.

Controller Synthesis

- ❖ **Controller Synthesis** is the process of designing the **control unit** that **drives** the data path.
- ❖ While the data path performs computations and moves data, the controller decides:
 - ❖ what happens, when it happens, and which hardware elements are active in each cycle.

What the Controller Does

The controller generates all **control signals** needed to execute the schedule:

1. Multiplexer control signals

Select which input goes into a functional unit
(e.g., select x or dx for a multiplier input)

2. Register enable signals

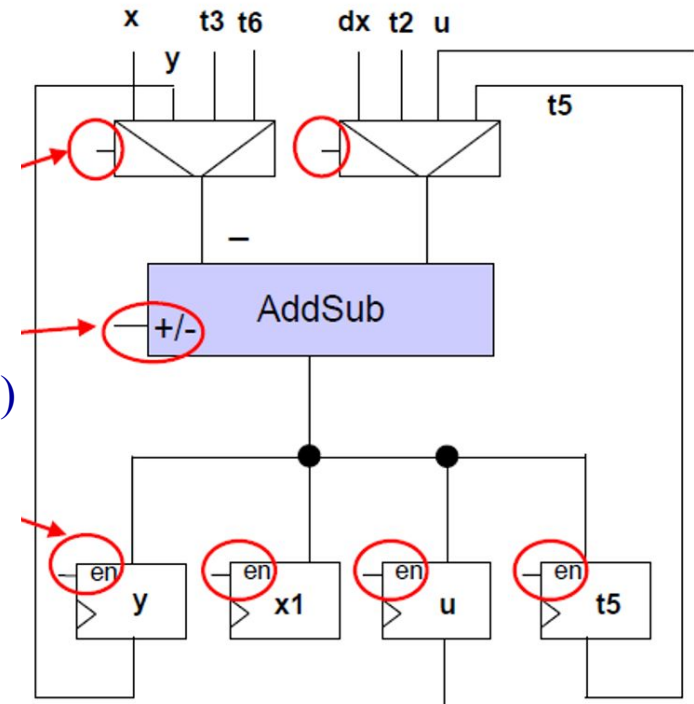
Decide **when registers store values**
(e.g., enable register t_1 in cycle 1, disable in others)

3. Operation mode control

Select operation type in shared resources
(e.g., AddSub block performs $+$ or $-$)

4. Resource activation

Decide which resource is active in which cycle



How Controller Synthesis Works

Step 1: Inputs to Controller Synthesis

Step 2: Build a Control Table (Controller Table)

Step 3: Define Control Actions Per Cycle

Step 4: Convert Control Table into FSM

Step 5: Add Control Flow

Step 6: Hardware Implementation

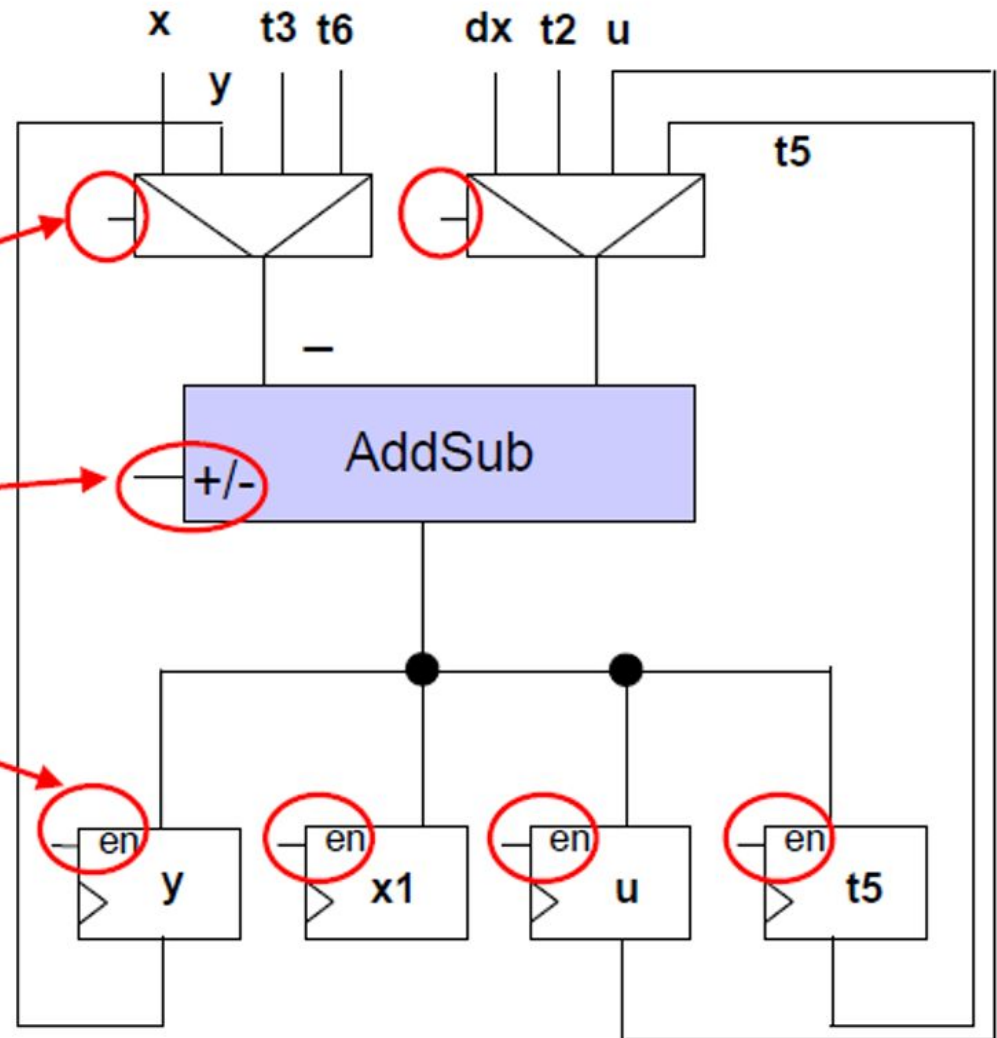
Step 7: Connect Controller to Datapath

Final Output: Complete RTL Architecture

Controller Synthesis

A controller must be developed / synthesized in order to

- generate the mux select signals
- define the operation mode of resources
- generate register enable signals



The controller drives the datapath by generating control signals that select data paths, choose operations, and enable storage allowing a scheduled algorithm to execute correctly in hardware.

Inputs to Controller Synthesis

Controller synthesis uses:

1. Schedule

→ tells *which operation runs in which cycle*

2. Allocation

→ tells *which resources exist*

3. Binding

→ tells *which operation uses which resource*

4. DFG

→ tells *data dependencies*

5. CFG (if branches exist)

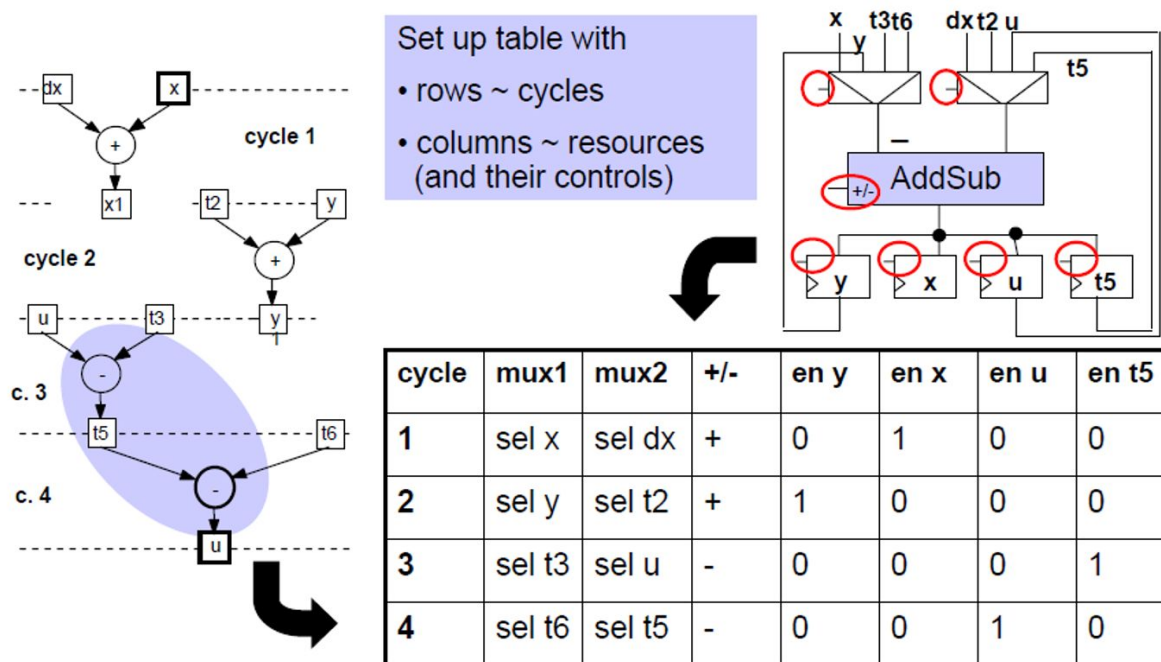
→ tells *control flow*

Build a Control Table (Controller Table)

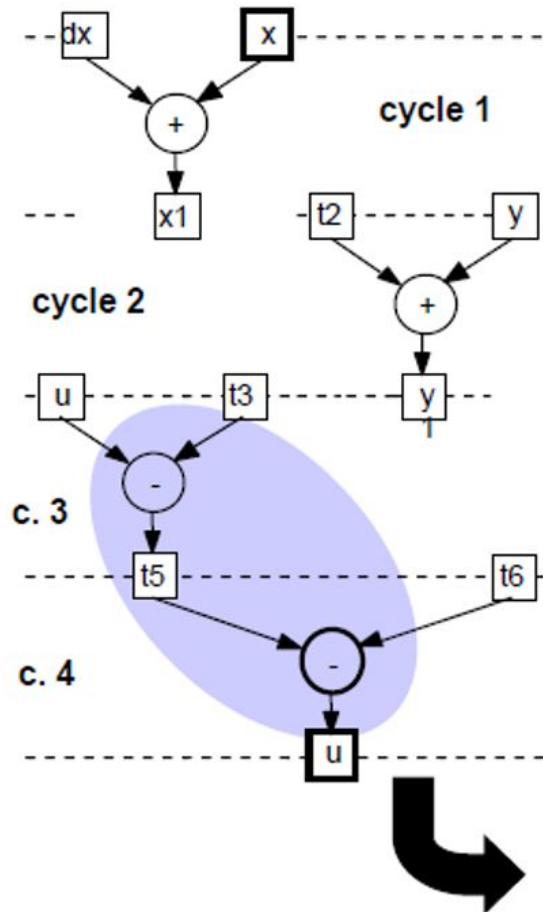
A table is created where:

- **Rows = cycles (control steps)**
- **Columns = datapath components (or their control inputs)**

Each cell defines what control signal value must be active in that cycle.

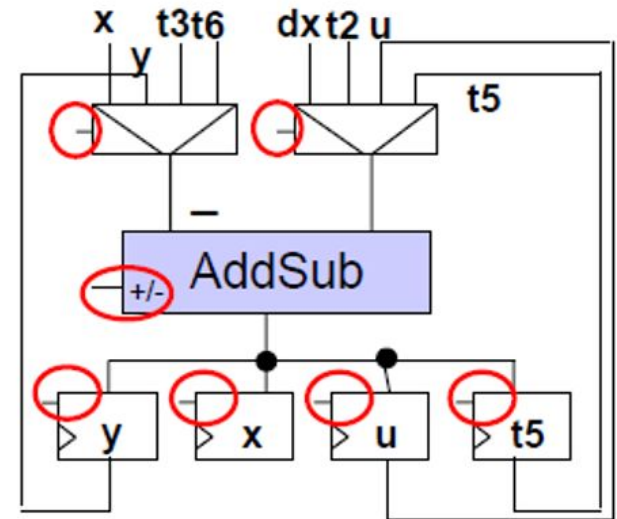


Controller Synthesis (Cont'd)



Set up table with

- rows ~ cycles
- columns ~ resources (and their controls)



cycle	mux1	mux2	+/-	en y	en x	en u	en t5
1	sel x	sel dx	+	0	1	0	0
2	sel y	sel t2	+	1	0	0	0
3	sel t3	sel u	-	0	0	0	1
4	sel t6	sel t5	-	0	0	1	0

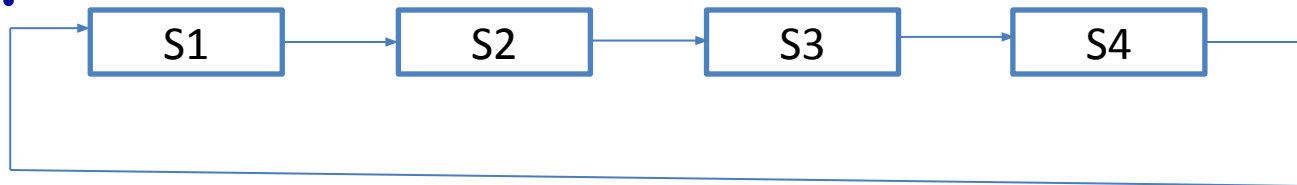
Convert Control Table into FSM

The controller table can be transformed into a **Finite State Machine (FSM)** that is easily implemented as a hardwired circuit at the gate level.

FSM Structure:

- ◆ Each cycle = one state
- ◆ Transitions = next cycle
- ◆ Outputs = control signals

Example:



Each state outputs:

- ☐ mux selects
- ☐ register enables
- ☐ operation modes
- ☐ resource enables

Encoding Step (Table → Binary Signals)

cycle	mux1	mux2	+/-	en y	en x	en u	en t5
1	sel x	sel dx	+	0	1	0	0
2	sel y	sel t2	+	1	0	0	0
3	sel t3	sel u	-	0	0	0	1
4	sel t6	sel t5	-	0	0	1	0

mux1: sel x → 00 sel y → 01 sel t3 → 10 sel t6 → 11	mux2: sel dx → 00 sel t2 → 01 sel u → 10 sel t5 → 11
+/- + → 0 - → 1	

FSM Construction

cycle	mux1	mux2	+/-	en y	en x	en u	en t5
1	sel x	sel dx	+	0	1	0	0
2	sel y	sel t2	+	1	0	0	0
3	sel t3	sel u	-	0	0	0	1
4	sel t6	sel t5	-	0	0	1	0

mux1: sel x → 00 sel y → 01 sel t3 → 10 sel t6 → 11	mux2: sel dx → 00 sel t2 → 01 sel u → 10 sel t5 → 11
+/- + → 0 - → 1	

State/Cycle	Table Row	Encoded	FSM State
S1/Cycle1	mux1 = sel x mux2 = sel dx +/- = + en y = 0 en x = 1 en u = 0 en t5 = 0	mux1.sel = 00 mux2.sel = 00 addsub.m = 0 regy.en = 0 regx.en = 1 regu.en = 0 regt5.en = 0	mux1.sel = 00 mux2.sel = 00 addsub.m = 0 regy.en = 0 regx.en = 1 regu.en = 0 regt5.en = 0
S2/Cycle2	mux1 = sel y mux2 = sel t2 +/- = + en y = 1 en x = 0 en u = 0 en t5 = 0	mux1.sel = 01 mux2.sel = 01 addsub.m = 0 regy.en = 1 regx.en = 0 regu.en = 0 regt5.en = 0	mux1.sel = 01 mux2.sel = 01 addsub.m = 0 regy.en = 1 regx.en = 0 regu.en = 0 regt5.en = 0

FSM Construction (Cont'd)

cycle	mux1	mux2	+/-	en y	en x	en u	en t5
1	sel x	sel dx	+	0	1	0	0
2	sel y	sel t2	+	1	0	0	0
3	sel t3	sel u	-	0	0	0	1
4	sel t6	sel t5	-	0	0	1	0

mux1:

sel x → 00
 sel y → 01
 sel t3 → 10
 sel t6 → 11

mux2:

sel dx → 00
 sel t2 → 01
 sel u → 10
 sel t5 → 11

+/-

+ → 0

- → 1

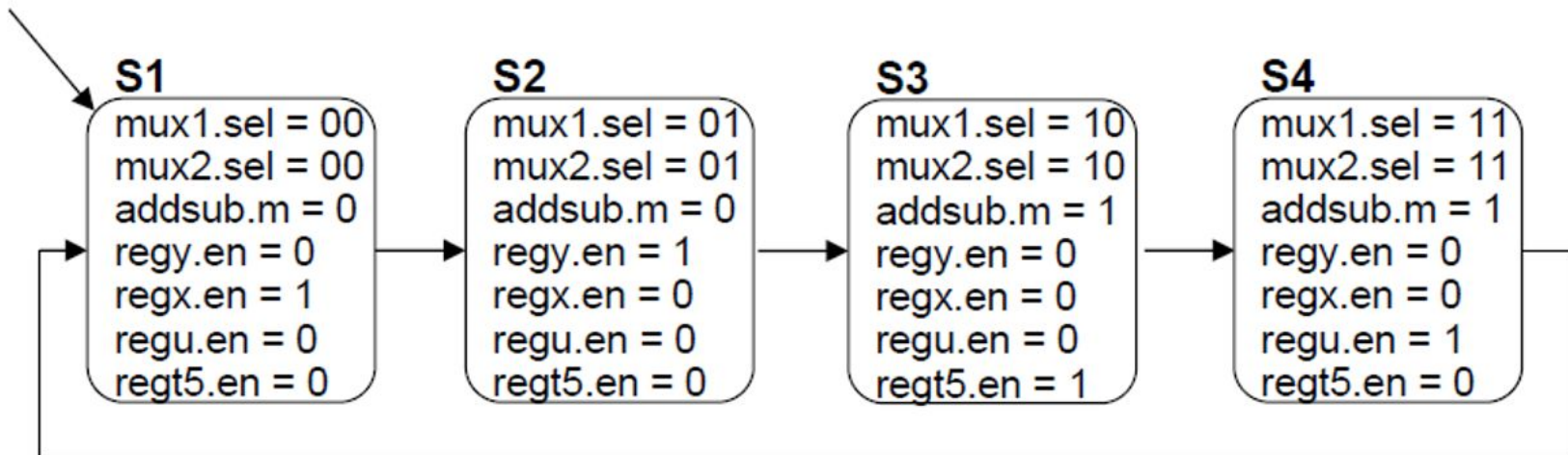
State/Cycle	Table Row	Encoded	FSM State
S3/Cycle3	mux1 = sel t3 mux2 = sel u +/- = - en y = 0 en x = 0 en u = 0 en t5 = 1	mux1.sel = 10 mux2.sel = 10 addsub.m = 1 regy.en = 0 regx.en = 0 regu.en = 0 regt5.en = 1	mux1.sel = 10 mux2.sel = 10 addsub.m = 1 regy.en = 0 regx.en = 0 regu.en = 0 regt5.en = 1
S4/Cycle4	mux1 = sel t6 mux2 = sel t5 +/- = - en y = 0 en x = 0 en u = 1 en t5 = 0	mux1.sel = 11 mux2.sel = 11 addsub.m = 1 regy.en = 0 regx.en = 0 regu.en = 1 regt5.en = 0	mux1.sel = 11 mux2.sel = 11 addsub.m = 1 regy.en = 0 regx.en = 0 regu.en = 1 regt5.en = 0

Controller Synthesis (Cont'd)

Translate table
into an FSM



cycle	mux1	mux2	+/-	en y	en x	en u	en t5
1	sel x	sel dx	+	0	1	0	0
2	sel y	sel t2	+	1	0	0	0
3	sel t3	sel u	-	0	0	0	1
4	sel t6	sel t5	-	0	0	1	0



Hardware Implementation

Hardwired FSM Controller

(Gate-level logic + flip-flops)

- Fast
- Area efficient
- Hard to modify
- Used in ASIC-style designs

Microprogrammed Controller

Horizontal Microprogramming

- ROM outputs = direct control signals
- Large ROM width
- Simple decoding

Vertical Microprogramming

- Encoded control signals
- Smaller ROM
- Extra decoding logic

ROM Address = cycle number

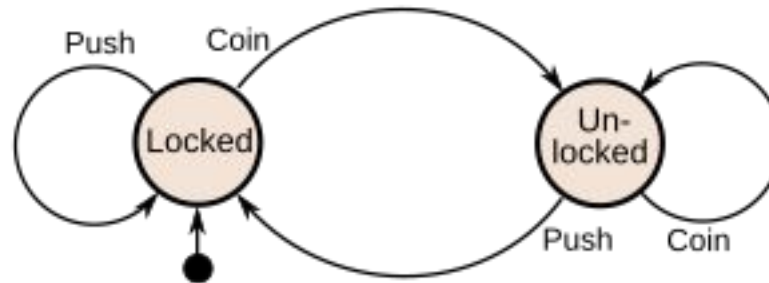
ROM Data = control word

Finite State Machine (FSM)

- ❖ A **Finite State Machine** (sometimes called a finite state automaton) is a computation model that can be implemented with hardware or software and can be used to simulate sequential logic and some computer programs. Finite state automata generate regular languages.
- ❖ Finite state machines can be used to model problems in many fields including mathematics, artificial intelligence, games, and linguistics.
- ❖ It represents a system with a limited number of states, transitioning between them in response to specific inputs, crucial for modeling sequential logic in hardware/software, seen in traffic lights or turnstiles, defined by states, inputs, transitions, and often visualized with state diagrams.

Finite State Machine (FSM)

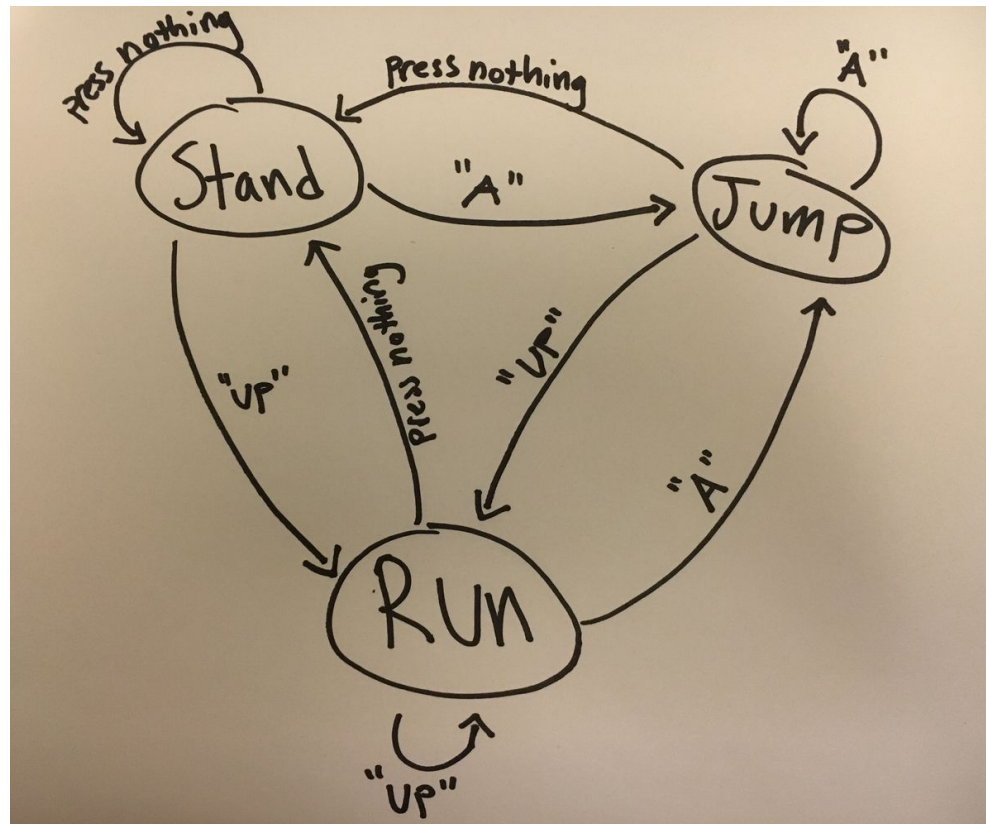
A system where particular inputs cause particular changes in state can be represented using finite state machines. This example describes the various states of a turnstile. Inserting a coin into a turnstile will unlock it, and after the turnstile has been pushed, it locks again. Inserting a coin into an unlocked turnstile, or pushing against a locked turnstile will not change its state.



A turnstile is a gate that controls pedestrian traffic, allowing only one person to pass at a time, often requiring payment (like a subway fare) or verification (like an ID or ticket) for entry, and is used in places like transit stations, stadiums, and buildings for security and access control, though the term also refers to a cybersecurity tool that replaces CAPTCHAs and a popular hardcore punk band.

Finite State Machine (FSM)

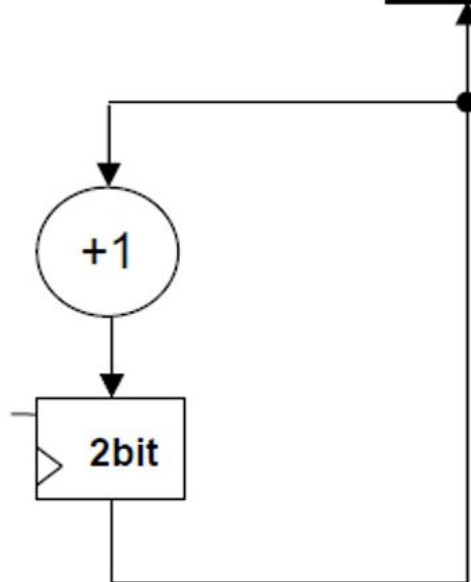
Here is a DFA diagram that describes a few simple moves that a character in a video game can do: stand, run, and jump. The buttons that a player can use to control this particular character are "Up," "A," or the player can press no button.



Controller Synthesis (Cont'd)

Alternative: ROM
table with counter

addr	mux1	mux2	+/-	en y	en x	en u	en t5
0	00	00	0	0	1	0	0
1	01	01	0	1	0	0	0
2	10	10	1	0	0	0	1
3	11	11	1	0	0	1	0



ROM content (word no. i) defines the value of the data path control signals in state $= i+1$

The controller is implemented as a **ROM (control memory)** + **counter (state register)**

- ✓ Each ROM word = one **control state**
- ✓ Each ROM output bit = one **control signal**

So instead of logic gates (FSM), we use **stored control words**.

ROM (Control Memory)

Stores control words

Each row = one cycle/state

Each column = one control signal

Field	Meaning
addr	ROM address (state number)
mux1	mux1 select bits
mux2	mux2 select bits
+/-	Add/Sub control
en y	enable register y
en x	enable register x
en u	enable register u
en t5	enable register t5

addr	mux1	mux2	+/-	en y	en x	en u	en t5
0	00	00	0	0	1	0	0
1	01	01	0	1	0	0	0
2	10	10	1	0	0	0	1
3	11	11	1	0	0	1	0



Counter and Adder

Counter (2-bit register)

- ❖ Generates ROM address
- ❖ Represents **current control state**
- ❖ Increments every clock cycle

+1 Adder

- ❖ Increments counter value by 1 each cycle
→ sequential state progression

Working Cycle-by-Cycle

Step 1: Initialization

Counter = 00
→ ROM address = 0
→ ROM word 0 is read

Step 2: ROM Read

For addr = 0 ROM outputs:

mux1 = 00
mux2 = 00
+/- = 0
en y = 0
en x = 1
en u = 0
en t5 = 0

Step 3: Counter Increment

counter = counter + 1

Step 4: Next ROM Address

addr = 1

mux1 = 01
mux2 = 01
+/- = 0
en y = 1
en x = 0
en u = 0
en t5 = 0

Working Cycle-by-Cycle

Counter	ROM Addr	Operation
00	0	cycle 1
01	1	cycle 2
10	2	cycle 3
11	3	cycle 4

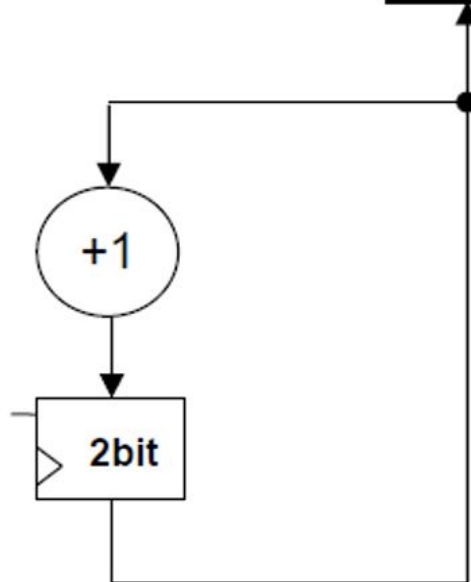
After 11 $\rightarrow$ next +1 $\rightarrow$ wraps to 00 (2-bit counter)
So, execution becomes cyclic:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0 \rightarrow 1 \rightarrow \dots$

Controller Synthesis (Cont'd)

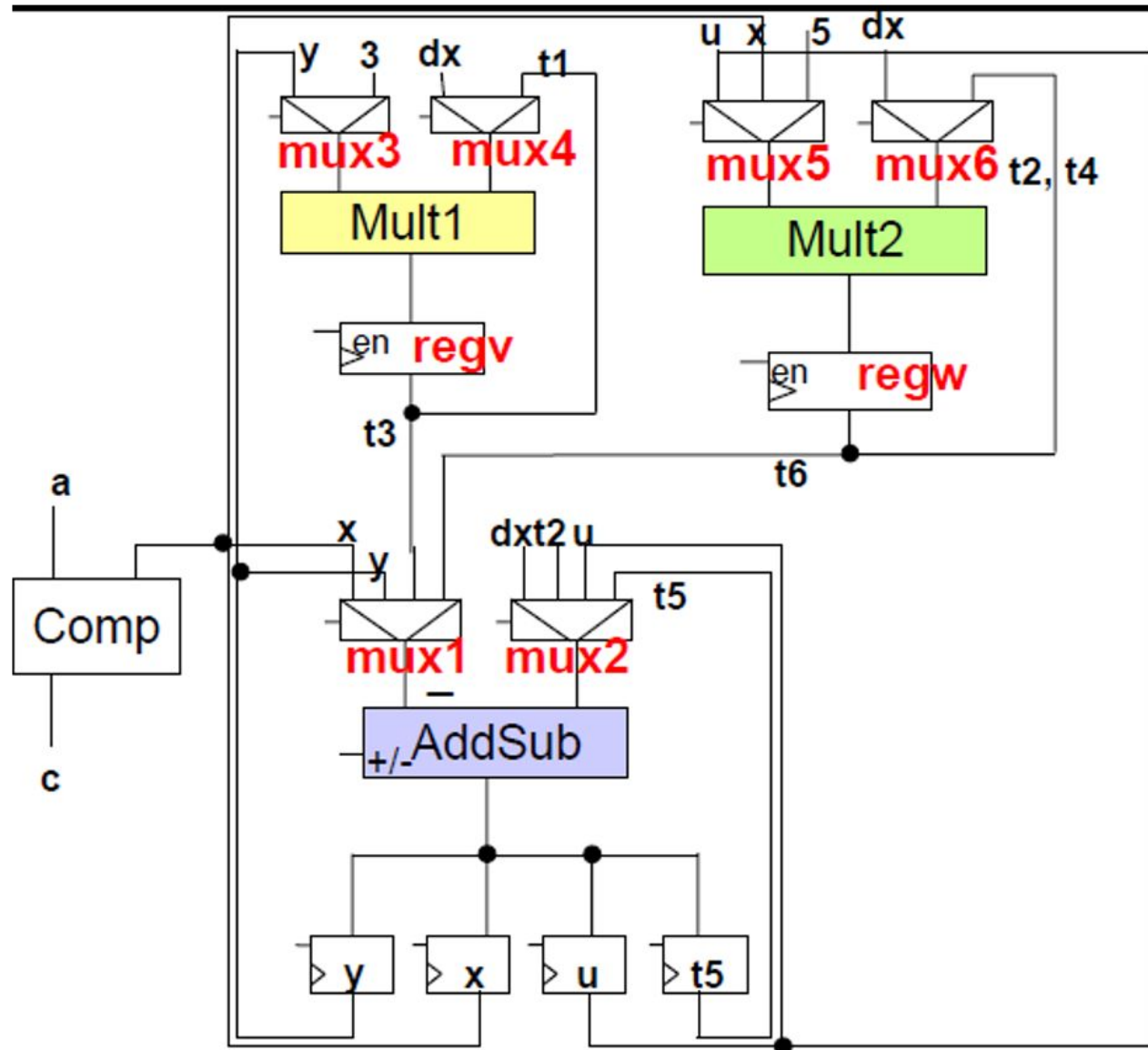
Alternative: ROM
table with counter

addr	mux1	mux2	+/-	en y	en x	en u	en t5
0	00	00	0	0	1	0	0
1	01	01	0	1	0	0	0
2	10	10	1	0	0	0	1
3	11	11	1	0	0	1	0



ROM content (word no. i) defines the value of the data path control signals in state $= i+1$

Complete Data Path



Complete Controller Table

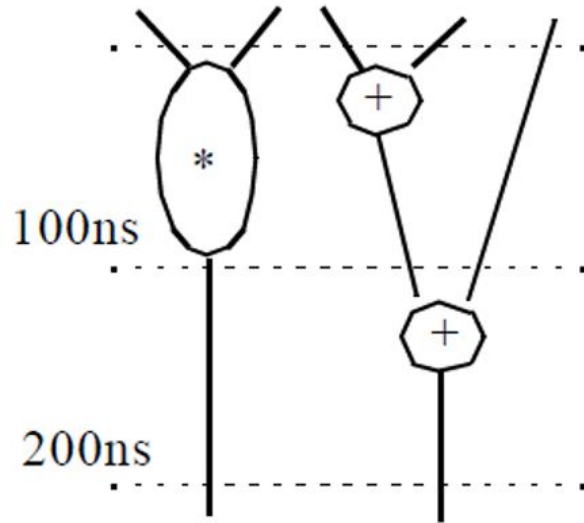
cycle	mux1	mux2	+/-	en y	en x	en u	en t5
1	sel x	sel dx	+	0	1	0	0
2	sel y	sel t2	+	1	0	0	0
3	sel t3	sel u	-	0	0	0	1
4	sel t6	sel t5	-	0	0	1	0

cycle	mux3	mux4	en v	mux5	mux6	en w
1	sel y	sel dx	1	sel u	sel dx	1
2	sel 3	sel t1	1	sel x	sel t_	1
3	--	--	0	sel 5	sel t_	1
4	--	--	0	--	--	0

Topics

- Mapping Specification → HW by High Level Synthesis / Design (architecture synthesis for data-dominated systems)
 - High Level Synthesis Overview
 - Basic Scheduling Algorithms: ASAP, ALAP
 - Resource-Constrained Scheduling
 - Time-Constrained Scheduling
 - **Advanced Scheduling: Operator Pipelining, Chaining & Multicycling**
 - Allocation, Binding, Data Path & Controller Generation
 - Pipelined Data Path Design
 - Pipeline Controller Design

No Chaining or Multicycling



(a) No chaining or multicycling

- ❖ Each operation must **fit in one clock cycle**
- ❖ Cycle time is determined by the **slowest operation**
- ❖ Example:
 - Multiplier = 100 ns
 - Adder = 40 ns
 - ➔ Clock cycle = **100 ns**
 - ➔ Adder wastes time (idle part of cycle)
 - ➔ Inefficient resource utilization

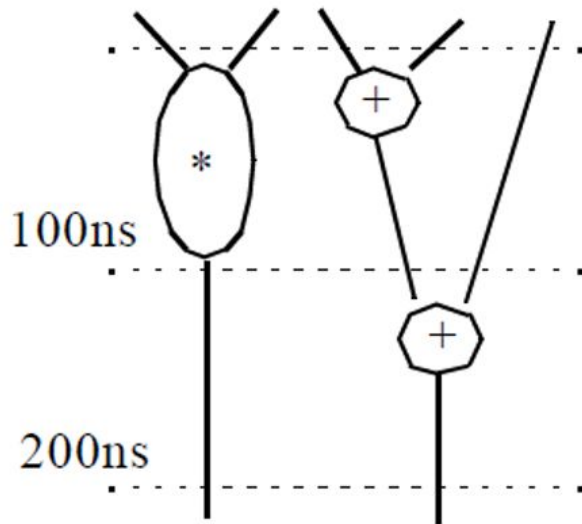
Taking into account that different operations require different amount of time; avoiding that the slowest operation dictates the performance of the complete data path

Chaining

Chaining is a technique in high-level synthesis where **multiple fast operations are executed sequentially within a single clock cycle**, without inserting registers between them.

- ❖ Operations are connected directly (combinationally).
- ❖ Only possible if the **total delay fits within one clock period**.
- ❖ No intermediate storage (no registers).
- ❖ Improves **latency** (fewer cycles).
- ❖ Increases **resource usage** (needs parallel hardware units).

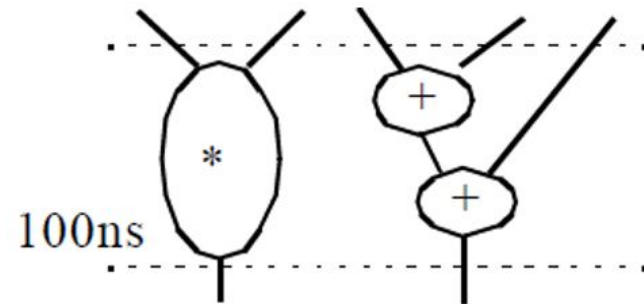
Chaining



(a) No chaining or multicycling

Taking into account that different operations require different amount of time; avoiding that the slowest operation dictates the performance of the complete data path

Adopted from Petru Eles, Zebo Peng



(b) Two chained additions

Chaining = multiple fast operations in one cycle

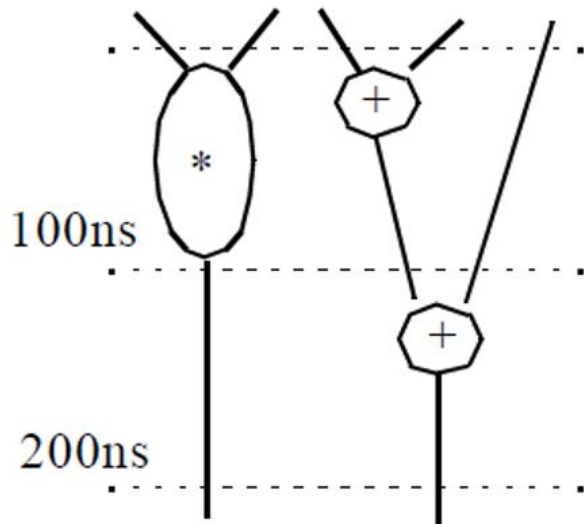
- ❖ Two adders chained in one cycle
- ❖ Still limited by **multiplier time**
- ❖ Cycle time = **100 ns**
- ❖ Latency reduced (two adds in one cycle instead of two cycles)
- ❖  But requires **two adders**
- ❖  Higher resource usage
- ❖  Faster execution
- ❖  More hardware

Multicycling

Multicycling is a technique where **slow operations are allowed to execute over multiple clock cycles**, instead of forcing them to complete in one cycle.

- ❖ Operation is **stretched across cycles**.
- ❖ Uses registers to store intermediate results.
- ❖ Reduces clock period (faster clock).
- ❖ Increases latency (more cycles per operation).
- ❖ Improves **resource efficiency**.

Multicycling



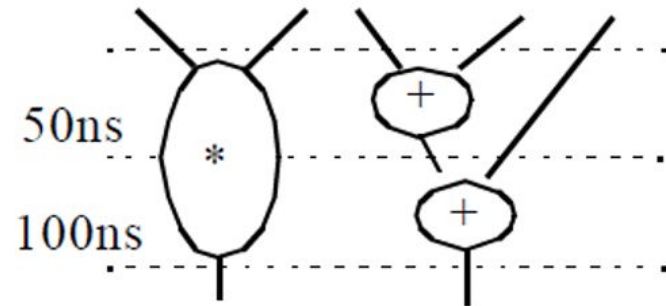
(a) No chaining or multicycling

Taking into account that different operations require different amount of time; avoiding that the slowest operation dictates the performance of the complete data path

Adopted from Petru Eles, Zebo Peng

Multicycling = slow operation spread across multiple cycles

- ❖ Multiplier split across 2 cycles
- ❖ New cycle time = **50 ns**
- ❖ Multiplier takes 2 cycles ($2 \times 50 \text{ ns} = 100 \text{ ns}$)
- ❖ Adders fit in single cycle ($40 \text{ ns} < 50 \text{ ns}$)
- ✓ Lower clock period
- ✓ Same hardware
- ✓ Better resource efficiency
- ✓ Better performance per cost



(c) A multicycle multiplication

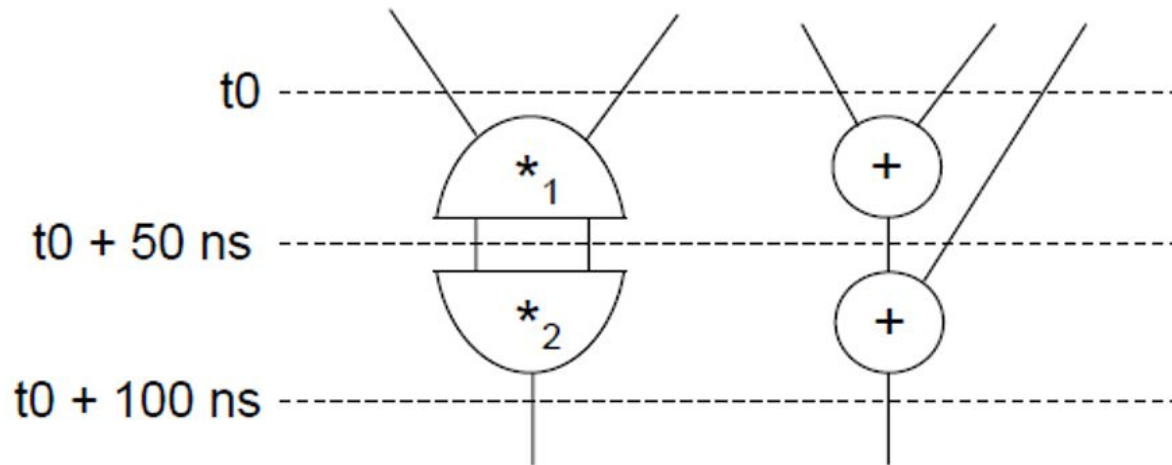
NOTE: Wire crossing cycle boundary => register

Pipelining

Pipelining is a technique where a hardware operator is **split into multiple stages with registers between them**, allowing **multiple operations to be processed in parallel across different stages**.

- ❖ Each stage does part of the computation.
- ❖ Registers separate stages.
- ❖ New operation starts every cycle.
- ❖ High throughput.
- ❖ Latency exists, but throughput is maximized.

Pipelining



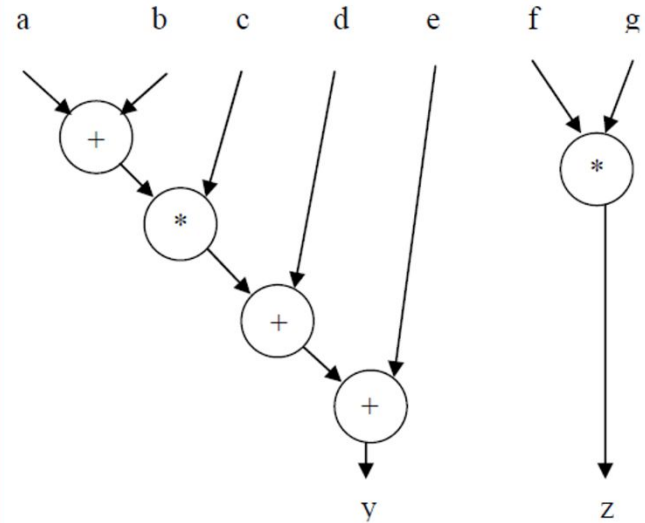
In this example, the multiplier is a *pipelined resource*.

- Multiplier is split into n (here: $n = 2$) stages
- Registers are introduced between the stages
- A new multiplication can begin in each cycle in the first stage, while the previous multiplication(s) is/are completed in the other pipeline stages

Exercise-01: List Scheduling with Multicycle Resources

Consider the following data flow graph:

- ▶ The following resources are available:
 - ▶ one adder which requires 40 ns per addition,
 - ▶ one multiplier which requires 80 ns per operation.
 - ▶ We use multicycling to achieve a cycle time of 50 ns.
- a) Perform a list scheduling of the data flow graph under the above assumptions.
- ▶ b) Try to find a better schedule with the same resources but less latency.



Solve yourself!

References

<https://brilliant.org/wiki/finite-state-machines/>